CASE STUDY

# Containerising CivicSense with Docker

Mohammad Iqbal Patan (24MCC20081) · Containerization With Docker (25CAH-734) · Guide: Sofia · Oct 2026

## BACKGROUND

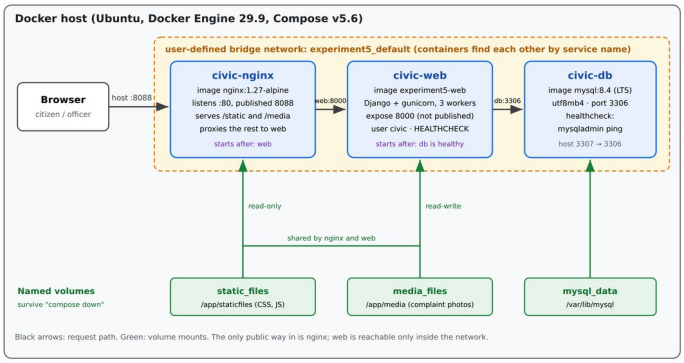
**CivicSense** is a civic complaint platform (Python, Django 6.1, MySQL) built in Experiment 5. Citizens report potholes, garbage or water problems in plain language; an NLP engine routes each complaint to a department, merges repeat reports of the same problem and ranks complaints by priority for municipal officers.

## THE CHALLENGE

- **Environment drift** – the MySQL driver needs a C compiler; Python versions and static files differed between machines.
- **Manual releases** – wait for MySQL, migrate, collect static, restart: one forgotten step took the portal down.
- **No resilience** – one process could not survive a crash, absorb a surge or be updated without downtime.

## THE DOCKER SOLUTION

- **Two-stage Dockerfile** – compiler only in a discarded builder stage (698 MB); shipped image **435 MB**, non-root user `civic`, built-in `HEALTHCHECK` on `/healthz`.
- **Entrypoint as code** – waits for MySQL, migrates, collects static files, then `exec gunicorn` as PID 1 for graceful shutdown.
- **Docker Compose** – MySQL 8.4 + Django/gunicorn + nginx with one command; private bridge network with DNS by service name, named volumes, start-up gated on `service_healthy`, only nginx published.
- **Docker Swarm** – the same image as 3 replicas behind a virtual IP on an overlay network, resource limits, start-first rolling updates with automatic rollback.
- **CI/CD** – GitHub Actions tests on real MySQL, then builds and smoke-tests every image; Kubernetes manifests for the cloud.




*Architecture of the Compose stack: only nginx is public; data lives in named volumes.*

## RESULTS (MEASURED ON 30 OCTOBER 2026, DOCKER 29.9)

| Test | Result |
|---|---|
| 39 automated tests inside the container (MySQL 8.4) | 39 / 39 passed in 85.8 s |
| Crash of the web container (SIGKILL) | auto-restart in 1.1 s, healthy in 11.3 s |
| Load balancing across 3 swarm replicas | 90 requests split exactly 30 / 30 / 30 |
| Scale 3 → 5 replicas | all healthy in 44 s |
| Replica killed during live traffic | replaced in 15.9 s — 522 OK, 0 failed |
| Rolling update / rollback under traffic | 94 s, 840 OK / 70 s, 622 OK — 0 failed |

## LESSONS LEARNED

- **Test the deployment, not just the code.** The real stack exposed defects invisible under `runserver`: data re-seeded on every automatic restart (120 → 240 complaints) and every login rejected by a CSRF origin check because the trusted origins lacked the published port. Both were found in container logs and fixed in configuration.
- **Health means "ready", not "started".** One cheap `/healthz` lets Docker, Swarm and CI decide when a container may take traffic.

## CONCLUSION

Docker turned CivicSense into one portable image that starts with a single command, heals itself after a crash, scales on demand and updates with **zero failed requests** – the same image running unchanged on a laptop, in Compose, in Swarm and in Kubernetes. Code and full report: **github.com/iPatan7/civicsense-docker**.